

Listing 7.1 Summing a list using a divide-and-conquer algorithm

```
def sum(ints: IndexedSeq[Int]): Int =
  if (ints.size <= 1)
    ints.headOption getOrElse 0
  else {
    val (l,r) = ints.splitAt(ints.length/2)
    sum(l) + sum(r)
  }
```

Divides the sequence in half using the `splitAt` function.

`IndexedSeq` is a superclass of random-access sequences like `Vector` in the standard library. Unlike lists, these sequences provide an efficient `splitAt` method for dividing them into two parts at a particular index.

`headOption` is a method defined on all collections in Scala. We saw this function in chapter 4.

Recursively sums both halves and adds the results together.

We divide the sequence in half using the `splitAt` function, recursively sum both halves, and then combine their results. And unlike the `foldLeft`-based implementation, this implementation can be parallelized—the two halves can be summed in parallel.

The importance of simple examples

Summing integers is in practice probably so fast that parallelization imposes more overhead than it saves. But simple examples like this are exactly the kind that are most helpful to consider when designing a functional library. Complicated examples include all sorts of incidental structure and extraneous detail that can confuse the initial design process. We're trying to explain the essence of the problem domain, and a good way to do this is to start with trivial examples, factor out common concerns across these examples, and gradually add complexity. In functional design, our goal is to achieve expressiveness not with mountains of special cases, but by building a simple and composable set of core data types and functions.

As we think about what sort of data types and functions could enable parallelizing this computation, we can shift our perspective. Rather than focusing on how this parallelism will ultimately be implemented and forcing ourselves to work with the implementation APIs directly (likely related to `java.lang.Thread` and the `java.util.concurrent` library), we'll instead design our own ideal API as illuminated by our examples and work backward from there to an implementation.

7.1.1 A data type for parallel computations

Look at the line `sum(l) + sum(r)`, which invokes `sum` on the two halves recursively. Just from looking at this single line, we can see that any data type we might choose to represent our parallel computations needs to be able to *contain a result*. That result will have some meaningful type (in this case `Int`), and we require some way of extracting this result. Let's apply this newfound knowledge to our design. For now, we can just

invent a container type for our result, `Par[A]` (for *parallel*), and legislate the existence of the functions we need:

- `def unit[A](a: => A): Par[A]`, for taking an unevaluated `A` and returning a computation that might evaluate it in a separate thread. We call it *unit* because in a sense it creates a unit of parallelism that just wraps a single value.
- `def get[A](a: Par[A]): A`, for extracting the resulting value from a parallel computation.

Can we really do this? Yes, of course! For now, we don't need to worry about what other functions we require, what the internal representation of `Par` might be, or how these functions are implemented. We're simply reading off the needed data types and functions by inspecting our simple example. Let's update this example now.

Listing 7.2 Updating `sum` with our custom data type

```
def sum(ints: IndexedSeq[Int]): Int =
  if (ints.size <= 1)
    ints.headOption.getOrElse 0
  else {
    val (l,r) = ints.splitAt(ints.length/2)
    val sumL: Par[Int] = Par.unit(sum(l))
    val sumR: Par[Int] = Par.unit(sum(r))
    Par.get(sumL) + Par.get(sumR)
  }
```

Computes the left half in parallel.

Computes the right half in parallel.

Extracts both results and sums them.

We've wrapped the two recursive calls to `sum` in calls to `unit`, and we're calling `get` to extract the two results from the two subcomputations.

The problem with using concurrency primitives directly

What of `java.lang.Thread` and `Runnable`? Let's take a look at these classes. Here's a partial excerpt of their API, transcribed into Scala:

```
trait Runnable { def run: Unit }

class Thread(r: Runnable) {
  def start: Unit
  def join: Unit
}
```

Begins running `x` in a separate thread.

Blocks the calling thread until `x` finishes running.

Already, we can see a problem with both of these types—none of the methods return a meaningful value. Therefore, if we want to get any information out of a `Runnable`, it has to have some side effect, like mutating some state that we can inspect. This is bad for compositionality—we can't manipulate `Runnable` objects generically since we always need to know something about their internal behavior. `Thread` also has the disadvantage that it maps directly onto operating system threads, which are a scarce resource. It would be preferable to create as many “logical threads” as is natural for our problem, and later deal with mapping these onto actual OS threads.

This kind of thing can be handled by something like `java.util.concurrent.Future`, `ExecutorService`, and friends. Why don't we use them directly? Here's a portion of their API:

```
class ExecutorService {
  def submit[A] (a: Callable[A]): Future[A]
}
trait Future[A] {
  def get: A
}
```

Though these are a tremendous help in abstracting over physical threads, these primitives are still at a much lower level of abstraction than the library we want to create in this chapter. A call to `Future.get`, for example, blocks the calling thread until the `ExecutorService` has finished executing it, and its API provides no means of *composing* futures. Of course, we can build the *implementation* of our library on top of these tools (and this is in fact what we end up doing later in the chapter), but they don't present a modular and compositional API that we'd want to use directly from functional programs.

We now have a choice about the meaning of `unit` and `get`—`unit` could begin evaluating its argument immediately in a separate (logical) thread,¹ or it could simply hold onto its argument until `get` is called and begin evaluation then. But note that in this example, if we want to obtain any degree of parallelism, we require that `unit` begin evaluating its argument concurrently and return immediately. Can you see why?²

But if `unit` begins evaluating its argument concurrently, then calling `get` arguably breaks referential transparency. We can see this by replacing `sumL` and `sumR` with their definitions—if we do so, we still get the same result, but our program is no longer parallel:

```
Par.get(Par.unit(sum(l))) + Par.get(Par.unit(sum(r)))
```

If `unit` starts evaluating its argument right away, the next thing to happen is that `get` will wait for that evaluation to complete. So the two sides of the `+` sign won't run in parallel if we simply inline the `sumL` and `sumR` variables. We can see that `unit` has a definite side effect, but only *with regard to* `get`. That is, `unit` simply returns a `Par[Int]` in this case, representing an asynchronous computation. But as soon as we pass that `Par` to `get`, we explicitly wait for it, exposing the side effect. So it seems that we want to avoid calling `get`, or at least delay calling it until the very end. We want to be able to combine asynchronous computations without waiting for them to finish.

¹ We'll use the term *logical thread* somewhat informally throughout this chapter to mean a computation that runs concurrently with the main execution thread of our program. There need not be a one-to-one correspondence between logical threads and OS threads. We may have a large number of logical threads mapped onto a smaller number of OS threads via thread pooling, for instance.

² Function arguments in Scala are strictly evaluated from left to right, so if `unit` delays execution until `get` is called, we will both spawn the parallel computation and wait for it to finish before spawning the second parallel computation. This means the computation is effectively sequential!